

Section 4: Architectural Patterns & Styles

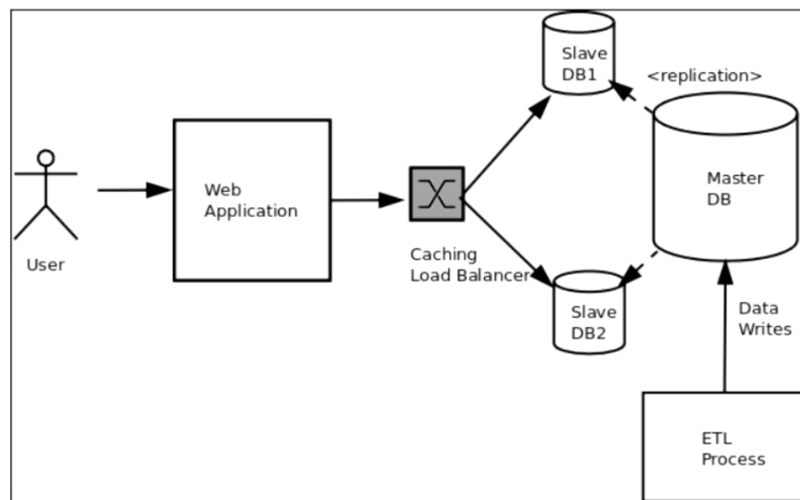
Software Architecture Patterns & Styles

In this section we will explore some of the most common architectural styles used in system design. These patterns are essential for creating scalable, maintainable and high performing software systems.

By the end of this session, you should have a clear understanding of when to use each of these architectural styles and how they can be applied to solve different technical challenges.

What is Software Architecture?

Software architecture is the high level structure of a software system. It defines the system's components, how they are organized and how they interact with each other. Think of it as the blueprint for the entire system, outlining its building blocks and how they fit together.



The architecture that we choose will directly impact several key aspects of our system. For instance it affects the below aspects:

- **Scalability:** It defines how well our system can handle growth of data and users.
- **Maintainability:** It defines how easy it is to modify or extend the system over time.
- **Performance:** It defines how fast and responsive the system is under load.

In short, your architectural choices can make or break your system's overall efficiency and behaviour.

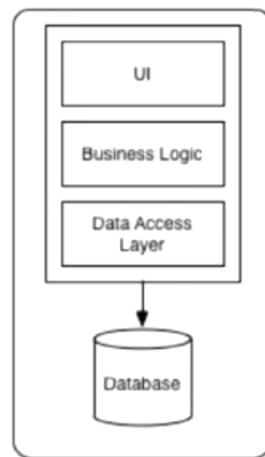
Common Architectural Styles

The below are the common architectural styles that are widely used in system design:

- **Monolithic Architecture:** In this style, the entire system is built as a single unified block. While it is easy to develop initially, it becomes difficult to scale and maintain as the system grows.
- **Layered (N-tier) Architecture:** This style separates the system into different layers such as presentation layer, business logic layer and data access layer. It improves maintainability but may introduce performance overheads due to multiple layers.
- **Client-Server Architecture:** In this style, there is a clear distinction between clients that make requests and servers that respond to those requests. This model is great for scalability, but this server can become a bottleneck sometimes.
- **Microservices Architecture:** Microservices breaks the system into smaller independent services that communicate with each other. This increases scalability and flexibility but can also introduce complexity in managing many services.
- **Event-Driven Architecture:** In this architecture, the system reacts to events such as user actions or external triggers. It is highly decoupled and scalable, but it can be harder to debug due to its asynchronous nature.

So each of these architectural styles has its strengths and weaknesses, and the choice of which one to use depends on the needs of your system. It is important to understand that the architecture sets the foundation of everything that follows in our system design.

Monolithic Architecture



Monolithic Architecture

This architecture is one of the simplest and most traditional architectural styles. In a monolithic system, all components of an application are tightly coupled and work together as a single unit. Monolithic architecture refers to a system where all functionalities are combined into one single unified code base. Every part of the application, from the user interface to business logic or database access code, everything exists within a single unit, making it very easier to develop initially.

Pros:

- As everything is in one place, it's easier to develop and deploy the application.
- There is no need to manage complex interaction between separate services, making it a good choice for smaller applications.
- For small scale applications, a monolithic architecture can be easier to manage because the tight coupling of components allows for faster communication between different parts of the system.

Cons:

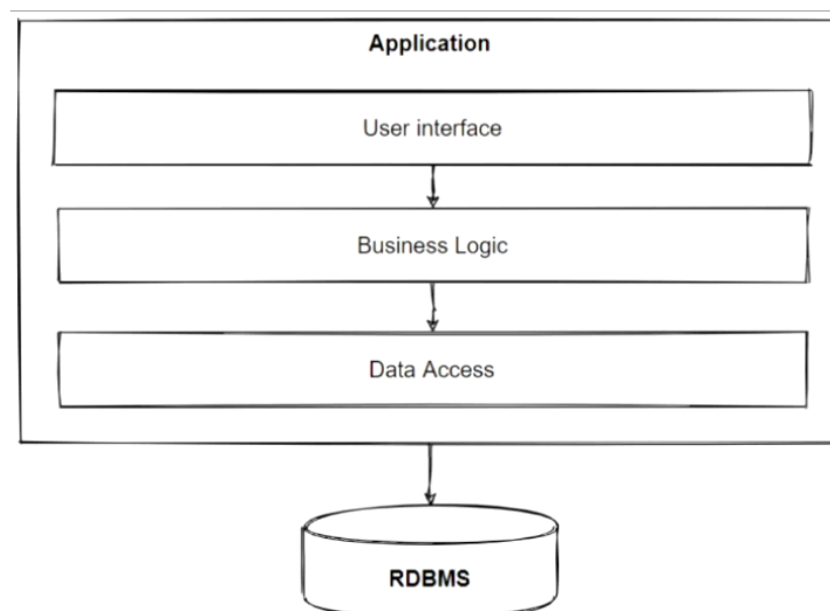
- It is very hard to scale, as everything is bundled together, scaling a monolithic application can be difficult. You have to scale the entire system even if only a part of it needs more resources.
- It is difficult to maintain, as the application grows over time, maintaining a large and often millions and millions of lines of code base becomes more complex. Changes to one part of the system can advertently affect other parts of the systems.
- There is a high risk of failure. Even if a single component fails, the entire system goes down. A bug in any part of the system can potentially cause a full system failure, which makes this architecture less resilient to more modular approaches.

Usecases:

- Small scale applications
- Startups and initial days
- Simple CRUD based applications

So to summarize, monolithic architecture can be a great choice for simple small scale applications where speed of development and simplicity are key. However as the system grows, it is important to be aware of the challenges that come with scaling, maintaining and ensuring the system's resilience.

Layered (N-Tier) Architecture



Layered Architecture which is also known as N-Tier architecture is an architecture style in which a system is divided into different layers, with each layer focusing on a specific aspect of the application. The idea here is to separate concerns which will help to manage the complexity. This architecture involves the splitting the application into distinct layers, each layer responsible for specific functionality. Common layers are presentation layer, which handles the user interaction, business logic layer which processes the core logic of the application and data access layer which manages the data access and storage. This separation of concern ensures that each latter is responsible for a specific task, making the system easier to understand and maintain.

Pros:

- It allows clear separation between different parts of the application. Each layer has a well-defined responsibility, which makes the system more modular and easier to maintain.
- It is easier to scale and maintain the application. For example, if the business logic becomes too complex, you can scale or update that layer independently of the presentation or data access layer.

Cons:

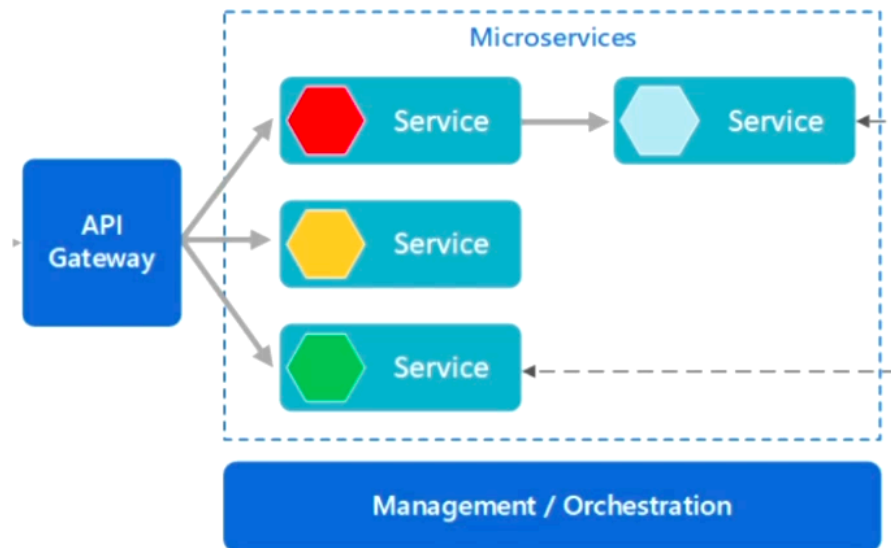
- Each layer introduces a bit of overhead, as data and requests must pass through multiple layers before reaching its final destination, which can actually result in performance issues, especially when our system grows.
- There is also an issue of tight coupling, while the layers are meant to be loosely coupled but in practice what happens is that they can sometimes become tightly coupled, meaning changes in one layer might require some changes in other layers also, which reduces flexibility and makes the future changes much more difficult.

Usecases:

- Large-enterprise applications
- CRM applications
- Banking and financial systems

So, a layered architecture provides a solid structure for managing complex applications and is a great choice for enterprise level systems. However, developers must be mindful of the performance overhead and the potential tight coupling between layers when scaling the system.

Microservices Architecture



This is one of the most popular architectural styles in modern system design, especially for large scale and cloud based applications. Microservices architecture is a design approach where the system is broken down into a collection of small independent services. Each of these services focus on a specific business capability or domain.

Pros:

- Microservices are loosely coupled, meaning they can be developed, deployed and scaled independently which also allows teams to work on different services without affecting each other, making deployment faster and more efficient.
- Each microservice can use a different technology stack that is best suited for the functionality of that service, they can be developed independently in any technology. For example, one service could be written in java while another service could be in python, allowing the teams to use the best tools for the job.
- The microservices are also independent, meaning the failure of one service doesn't necessarily bring down the whole system. This provides better fault tolerance as each service can be isolated and handled independently.

Cons:

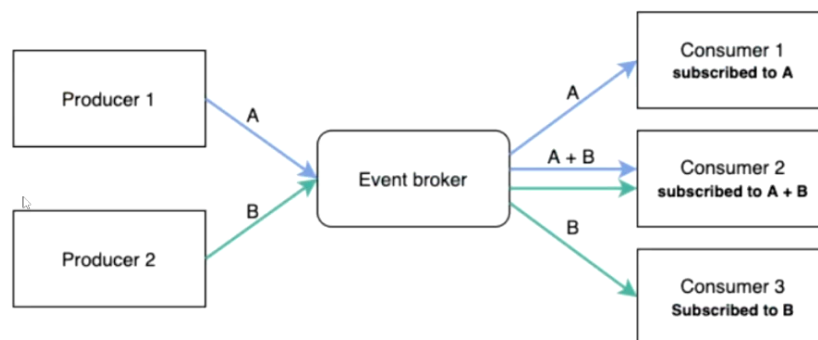
- With many small services communicating over a network, managing inter-service communication becomes very complex. Managing network failures, ensuring data consistency, and managing service dependencies can become a challenge.
- We need to have very robust DevOps and automation pipelines. Microservices require strong DevOps practices and automation pipeline for continuous integration, continuous deployment, and monitoring. Without them managing these services becomes very overwhelming and error-prone also.

Usecases:

- Large-scale applications (e.g., E-commerce platforms).
- Cloud-based systems (AWS, Azure, GCP).

So to summarize, while microservices architecture brings flexibility and scalability, it also introduces complexity that needs to be managed very carefully. If we utilize the microservices right, they can provide a robust, scalable solution for large and complex applications. However, the overhead of managing multiple services and ensuring smooth communication requires very careful planning and strong DevOps practices.

Event-Driven Architecture



Event-driven architecture is a very powerful architectural style designed to support highly decoupled and reactive systems. This style is becoming very popular day by day, especially for real-time applications. Real-time systems often require event-driven architecture, which provides an approach where components in a system communicate by emitting and consuming events, which are nothing but messages going from one service to another service. But rather than making a direct function call of another service, an event will be sent to the other service. An event could be anything from a user action, a change in data or a trigger within the system. This setup actually enables loose coupling between components meaning services don't need to be aware of each other's internal working and this can also be used along with microservices meaning communication between microservices can be event-driven.

Pros:

- Components communicate via events, they don't need to know about each other's internal logic which creates a more flexible, maintainable system where components can evolve independently without affecting one another.
- Events are processed asynchronously, which allows the system to handle workflows that might take time or require multiple steps without blocking their operations. This is especially useful in environments where speed and responsiveness are critical.
- The components are loosely coupled and communicate asynchronously making them highly scalable. For example, when traffic increases, you can add more consumers to process events without affecting the producers of those events.

Cons:

- Debugging and tracing becomes much more complex as events flow through a distributed system, tracing the path of an event can be very difficult. It can be harder to debug issues or understand how data flows through the system, making it necessary to have good logging and monitoring in place.
- Data consistency can be a challenge as the events are processed asynchronously. Ensuring the data stays consistent across multiple services requires careful management of events and there has to be a concept of eventual consistency, which is a little tricky to implement correctly.

Usecases:

- Real-time systems (e.g., live notifications, alerts).
- IoT applications
- Financial trading platforms

So to summarize, event-driven architecture allows for a highly flexible, scalable system, especially in the environments that require asynchronous processing or real-time communication. However, managing complexity and ensuring data consistency in such systems can be challenging. But with the right tools and strategies in place, this architecture can greatly improve the responsiveness and scalability of our system.

Factors Influencing Architecture Selection

Choosing the right architecture style for your system is very important. Choosing the right one isn't just about selecting the latest trend or the most popular style. It is about making sure that the architecture aligns with specific needs of our application. So let's look at some key factors that should guide our decision-making process when selecting an architecture.

- **Business Needs:** What problem are we solving? What are the specific business goals we need to achieve with this system? Your architecture must support these goals. For instance, if your business requires quick iterations and a lot of flexibility, microservices might be a good choice. If you need something simpler and quick to build, a monolithic approach could be better.
- **Scalability:** How much traffic and data should the system be able to handle? If you expect rapid growth or high traffic you will need an architecture that can scale easily both vertically and horizontally.
- **Performance:** How fast should your system respond to users? In certain applications like gaming, financial trading, or real-time analytics, performance is non-negotiable. Architectures like microservices can be optimized for performance in high-demand environments, but they can also introduce latency due to inter-service communication.
- **Maintainability:** How easy is it to make updates, fix bugs, and evolve the system over time? Maintainability is how easy it will be to make system updates, fix bugs, and evolve the system over time. So we should also keep this in mind selecting the approach that we want to use for our application.

So to summarize, selecting the right architecture requires careful considerations of factors like business needs, scalability, performance, and maintainability. These all should be weighed in to ensure that the architecture aligns with the goals and challenges of the system. A well chosen architecture will provide the foundation of a successful, scalable and maintainable system.

Summary & Key Takeaways

- Understand different architectural styles: Monolithic, Layered, Client-Server, Microservices, Event-Driven.
- Each style has its pros, cons, and best-use cases.
- Architecture decisions impact scalability, performance, maintainability, and system behavior.
- Choose architecture based on business needs, technical requirements, and scalability goals.
- Whats next:
 - Multi-Tier Architecture

Interview Questions

1. What is the difference between Monolithic and Microservices architecture?
2. When would you choose a Layered architecture over Microservices?
3. What are the pros and cons of Event-Driven Architecture?
4. How does architecture impact system scalability and performance?
5. How do business requirements influence architectural decisions?
6. Can you explain the key differences between Layered and Client-Server architectures?
7. What are the main challenges in maintaining a Monolithic system as it grows?
8. How do you decide between Monolithic and Microservices for a new system?
9. What role does fault tolerance play in Microservices architecture?
10. How do event-driven systems handle real-time data?